Unit III Searching and Sorting

Prof. Pranjal Pandit

Searching and Sorting

Searching:

- sequential/linear search,
- sentinel search,
- binary search,
- Fibonacci search,
- indexed sequential search.

Sorting:

- internal vs external sorting;
- sort order, stability, efficiency and number of passes;
- comparison-based methods (Bubble, Insertion, Selection, Quick, Shell);
- non-comparison-based methods (Radix, Counting, Bucket);
- comparison of all methods and their complexities.

Searching Techniques in Data Structures

- Searching is the process of **finding whether a particular element (called the key or target) exists in a collection of data**.
- If the element is found, its position (index) is returned; otherwise, the search reports that the element is not present.

Sequential (Linear) Search

Definition: Sequential Search (also called **Linear Search**) is the simplest searching algorithm. It checks each element **one by one** from the beginning until the required element is found or the list ends.

It can be used on:

- Sorted arrays
- Unsorted arrays

No sorting is required.

Algorithm (Linear Search)

- Start from the first element.
- Compare it with the target.
- If they are equal, return the index.
- Otherwise move to the next element.
- Repeat until:
  - Target is found, or
  - End of array is reached.

Example 1

Array

```
Index : 0 1 2 3 4 5
Array : 12 25 8 40 17 31
```

Search = **17**

Step-by-Step

Step	Current Element	Comparison	Result
1	12	12==17?	No
2	25	25==17?	No
3	8	8==17?	No
4	40	40==17?	No
5	17	17==17?	Found

Position = **Index 4**

Write code for Linear Search

Time Complexity

Case	Complexity
Best	O(1)
Average	O(n)
Worst	O(n)

Advantages

- Very easy to implement.
- Works on sorted and unsorted arrays.
- No preprocessing required.

Disadvantages

- Slow for large datasets.
- Many comparisons may be needed.

Sentinel Search

Definition: Sentinel Search is an **improved version of Linear Search**.

Instead of checking both

- "Is the element equal?"
- "Have we reached the end?"

on every iteration, Sentinel Search places the target value temporarily at the end of the array (called the **sentinel**).

This removes one comparison inside the loop. Hence it performs slightly faster.

Idea

Suppose

Array: 10 25 30 15 8

Search = 15

Normally Linear Search checks

- i<n ?
- arr[i]==key ?

every iteration.

Sentinel Search temporarily changes the last element into the search key.

Example

Original 10 25 30 15 8

Suppose searching **50**

Temporarily 10 25 30 15 50

Now the search simply continues until **50** is encountered.

Finally, restore the last element. determine whether the match occurred before the last position (found) or only at the sentinel (not found).

Example

Original Array 20 35 15 40 60

Search = **15**

Last element = 60

Replace last element by 15: 20 35 15 40 15

Now compare

20 → No

35 → No

15 → Found

Restore

20 35 15 40 60

Since found before the last position, the element exists.

Example (Not Found)

Array 5 12 8 20 9

Search = **15**

Temporary array 5 12 8 20 15

Search

5

12

8

20

15

The search stops at the sentinel.

Restore last element.

Original 5 12 8 20 9

Since the only match was the sentinel, the element is **not present**.

Time Complexity

Although the asymptotic complexity is the same as Linear Search, Sentinel Search reduces one comparison per loop iteration, so it is slightly faster in practice.

Case	Complexity
Best	O(1)
Average	O(n)
Worst	O(n)

Linear Search

Suppose the array is

10 20 30 40 50

Search = **50**

The algorithm is usually written as:

```
while(i < n)
{
   if(arr[i] == key)
      return i;
   i++;
}
```

For **every iteration**, two checks are made.

Iteration 1

i < n ? ✓

arr[0]==50 ? ✗

2 comparisons

Total= 5 boundary checks + 5 key comparisons=10 comparisons

Sentinel Search

Now take the same array.

10 20 30 40 50

Suppose we are searching **35**.

Replace the last element with 35.

10 20 30 40 35

Now the algorithm becomes

```
while(arr[i] != key)
{
   i++;
}
```

Notice something?

There is **no i < n check** inside the loop.

Only one comparison is done.

Total comparisons→5 key comparisons

Sentinel Search

Advantages

- Faster than ordinary linear search.
- Fewer comparisons.
- Simple implementation.

Disadvantages

- Requires temporarily modifying the array.
- Not suitable when the array must remain unchanged (unless you restore the last element).

Linear Search vs Sentinel Search

Feature	Linear Search	Sentinel Search
Boundary check each iteration	Yes	No
Modifies array	No	Temporarily
Time Complexity	O(n)	O(n)
Comparisons	More	Fewer
Speed	Slightly slower	Slightly faster

Write code for Sentinel Search

Binary Search

Binary Search is a searching technique that repeatedly divides the search space into two halves.

Important: Binary Search works **only on sorted arrays**. If the array is not sorted, Binary Search will produce incorrect results.

Working Principle

Suppose

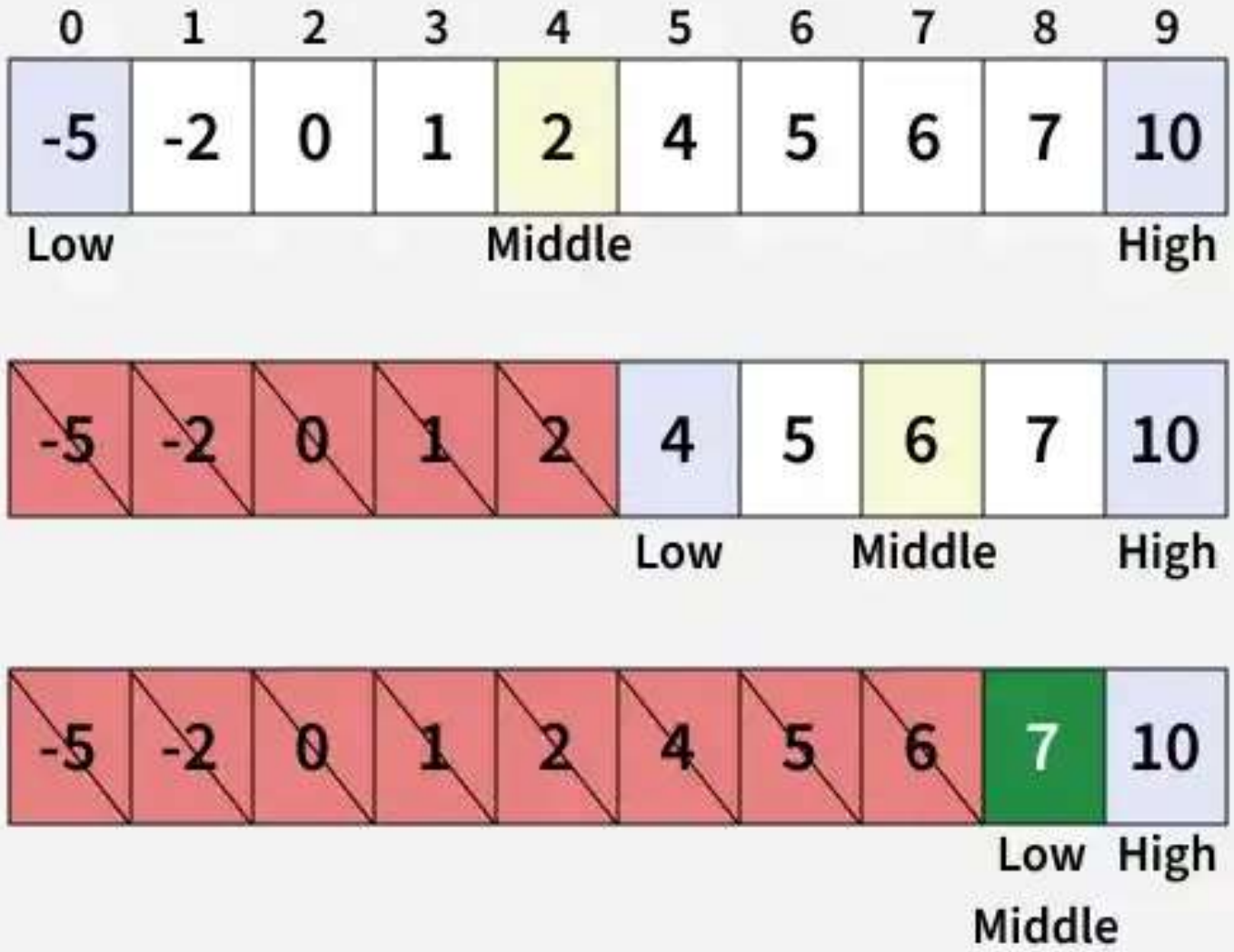
1 3 5 7 9 11 13 15

Search = **11**

Instead of checking every element,

- Binary Search checks the middle element.
- If target equals middle → Found.
- If target is smaller → Search left half.
- If target is larger → Search right half.

Repeat until found.



Searching for 7 : We repeatedly go to mid. If mid is smaller, then right half. Else left half

Time Complexity

This makes Binary Search much faster than Linear Search for large datasets.

Case	Complexity
Best	O(1)
Average	O(log n)
Worst	O(log n)

Advantages

- Very fast for large sorted arrays.
- Performs very few comparisons.
- Efficient with large datasets.

Disadvantages

- Works only on sorted data.
- Sorting may take extra time if the data is initially unsorted.

Comparison of All Three Searching Techniques

Feature	Sequential Search	Sentinel Search	Binary Search
Requires Sorted Array	No	No	Yes
Works on Unsorted Data	Yes	Yes	No
Method	Check one by one	Linear search with sentinel	Divide array into halves
Best Case	O(1)	O(1)	O(1)
Average Case	O(n)	O(n)	O(log n)
Worst Case	O(n)	O(n)	O(log n)
Suitable for Small Arrays	Yes	Yes	Yes
Suitable for Large Arrays	No	No	Yes (if sorted)

Write code for Binary Search

CODE

```
int main()
{
    int n, key, choice;

    cout << "Enter number of elements: ";
    cin >> n;
    int arr[100];
    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    cout << "\nEnter element to search: ";
    cin >> key;
    cout << "\n===== SEARCH MENU =====\n";
    cout << "1. Linear Search\n";
    cout << "2. Sentinel Search\n";
    cout << "3. Binary Search\n";
    cout << "Enter your choice: ";
    cin >> choice;

    int pos;
```

```
 switch (choice)
   {
      case 1:
         pos = linearSearch(arr, n, key);
         break;
      case 2:
         pos = sentinelSearch(arr, n, key);
         break;
      case 3:
         cout << "\nNote: Binary Search requires a SORTED array.\n";
         pos = binarySearch(arr, n, key);
         break;
      default:
         cout << "Invalid Choice!";
         return 0;
   }
   if (pos != -1)
      cout << "\nElement found at index " << pos;
   else
      cout << "\nElement not found.";
   return 0;
}
```

```
#include <iostream>
using namespace std;

// Linear Search
int linearSearch(int arr[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == key)
            return i;
    }
    return -1;
}
```

```
// Sentinel Search
int sentinelSearch(int arr[], int n, int key)
{
   int last = arr[n - 1];      // Save last element
   arr[n - 1] = key;           // Place sentinel

   int i = 0;
   while (arr[i] != key)
   {
      i++;
   }
arr[n - 1] = last;          // Restore last element

   if (i < n - 1 || last == key)
      return i;
   else
      return -1;
}
```

```
// Binary Search (Array must be sorted)
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;

        else if (key < arr[mid])
            high = mid - 1;

        else
            low = mid + 1;
    }

    return -1;
}
```

Fibonacci Search

Fibonacci Search is a searching algorithm used to find an element in a **sorted array**. It works similarly to Binary Search but instead of dividing the array into two equal halves, it divides the array using **Fibonacci numbers**.

It uses the Fibonacci sequence:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Why use Fibonacci Search?

- Works only on **sorted arrays**
- Uses only addition and subtraction (no division)
- Useful when division operation is expensive
- Time Complexity: **O(log n)**

Idea Behind Fibonacci Search

Suppose the array size is **15**.

Find the smallest Fibonacci number greater than or equal to 15.

Fibonacci numbers:

0 1 1 2 3 5 8 13 21

Smallest ≥15 is **21**.

The algorithm compares elements at Fibonacci distances rather than the middle.

Example

Consider the sorted array

Index : 0 1 2 3 4 5 6 7 8 9

Array : 10 22 35 40 45 50 80 82 85 90

Search for **80**

Step 1

Size = 10

Smallest Fibonacci ≥10

0 1 1 2 3 5 8 13

FibM =13

FibMm1 =8

FibMm2 =5

Offset = -1

Step 2

Calculate

i = min(offset + FibMm2, n-1)

i = min(-1+5,9)

i =4

Compare

A[4]=45

Since

80 >45

Move to right side.

Update Fibonacci numbers.

Array : 10 22 35 40 45 50 80 82 85 90

Search for **80**

Smallest Fibonacci ≥10

0 1 1 2 3 5 8 13

Step 3

Now

FibM =8

FibMm1=5

FibMm2=3

offset=4

New index

i=min(4+3,9)

i=7

A[7]=82

Since

80<82

Move left.

Step 4

Update

FibM=3

FibMm1=2

FibMm2=1

(Offset same)

Now

i=min(4+1,9)

i=5

A[5]=50

80>50

Move right.

Step 5

Update

offset=5

FibM=2

FibMm1=1

FibMm2=1

Now

i=min(5+1,9)

i=6

A[6]=80

Found.

Fibonacci Search Algorithm

1. Find the smallest Fibonacci number greater than or equal to n.
2. Initialize offset = -1.
3. While Fibonacci number >1

    i=min(offset+FibMm2,n-1)

    If key>A[i]

        Move right

    Else if key<A[i]

        Move left

    Else

        Return index

4. Check the last remaining element.
5. If found return index
6. Else return -1.

```
int fibonacciSearch(int arr[], int n, int key)
{
    int fibMMm2 = 0;
    int fibMMm1 = 1;
    int fibM = fibMMm1 + fibMMm2;

    while (fibM < n)
    {
        fibMMm2 = fibMMm1;
        fibMMm1 = fibM;
        fibM = fibMMm1 + fibMMm2;
    }
    int offset = -1;
```

```
while (fibM > 1)
  {
     int i = min(offset + fibMMm2, n - 1);

     if (arr[i] < key)
     {
        fibM = fibMMm1;
        fibMMm1 = fibMMm2;
        fibMMm2 = fibM - fibMMm1;
        offset = i;
     }
     else if (arr[i] > key)
     {
        fibM = fibMMm2;
        fibMMm1 = fibMMm1 - fibMMm2;
        fibMMm2 = fibM - fibMMm1;
     }
     else
     {
        return i;
     }
  }
```

Time Complexity

Space Complexity = **O(1)**

Why O(log n)???

Case	Complexity
Best	O(1)
Average	O(log n)
Worst	O(log n)

Advantages

- Faster than Linear Search.
- No division operation.
- Efficient for large sorted arrays.

Disadvantages

- Works only on sorted arrays.
- More complicated than Binary Search.

Indexed Sequential Search (Index Sequential Search)

What is Indexed Sequential Search?

Indexed Sequential Search is an improved version of **Linear Search**. Instead of searching every element, an **index table** is created.

Each index stores:

- The first (or highest) key of a block
- The starting position of that block

The search happens in **two phases**:

- Search the index table to find the correct block.
- Perform a linear search only within that block.

This is much faster than searching the entire array.

Example

Suppose we have the following sorted array:

Index : 0 1 2 3 4 5 6 7 8 9 10 11

Array : 5 8 10 15 18 22 27 30 35 40 45 50

Divide into blocks of **3** elements.

Block 1: 5 8 10

Index entry 10

Block 2: 15 18 22

Index entry 22

Block 3: 27 30 35

Index entry 35

Block 4: 40 45 50

Index entry 50

Index Table:

Block	Maximum Value	Start Index
1	10	0
2	22	3
3	35	6
4	50	9

Suppose we want to search **30**.

Step 1: Search the Index Table

Compare 30 with index values:

- 30 > 10
- 30 > 22
- 30 ≤ 35 → **Block 3** is selected.

Step 2: Linear Search in Block 3

Block 3 contains:

27 30 35

Compare:

- 27 ≠ 30
- 30 = 30 → **Found**

Only 2 comparisons were needed inside the block instead of searching all 12 elements.

Example for practice

Suppose we have the following sorted array: Search 63

Index	Value
0	10
1	15
2	22
3	28
4	35
5	41
6	48
7	55
8	63
9	70
10	76
11	82
12	90
13	95
14	100

Indexed Sequential Search Algorithm

1. Divide the sorted array into fixed-size blocks.

2. Create an index table containing:
   - Maximum (or first) element of each block
   - Starting index of each block

3. Compare the key with index entries.

4. Select the appropriate block.

5. Perform linear search inside that block.

6. If found, return the index.

7. Otherwise, return -1.

```
#include <iostream>
using namespace std;
int indexedSequentialSearch(int arr[], int n, int key, int blockSize)
{
    int start = 0;
    while (start < n)
    {
        int end = min(start + blockSize - 1, n - 1);

        if (key <= arr[end])
        {
            for (int i = start; i <= end; i++)
            {
                if (arr[i] == key)
                    return i;
            }
            return -1;
        }
        start += blockSize;
    }
    return -1;
}
```

Time Complexity

Let: **n** = total number of elements **b** = block size

Searching the index table: **O(n / b)**

Linear search within a block: **O(b)**

Total time: **O(n / b + b)**

If **b ≈ √n**, then: **Time Complexity = O(√n)**

Space Complexity = **O(n / b)** for storing the index table.

Advantages

- Faster than Linear Search for large sorted files.
- Easy to implement.
- Reduces the number of comparisons by limiting the search to one block.

Disadvantages

- Requires a sorted array.
- Needs extra memory for the index table.
- Less efficient than Binary Search for in-memory arrays, but useful for large files or databases where data is accessed in blocks.

A sorted array contains the following elements:

Index	0	1	2	3	4	5	6	7	8	9	10	11
Array	5	12	18	23	27	34	39	45	52	60	68	75

The array is divided into blocks of **3 elements**.

Index Table

Block	Starting Key	Starting Index
1	5	0
2	23	3
3	39	6
4	60	9

Questions

1. Search for **52** using Indexed Sequential Search.
2. Show every comparison performed.
3. State whether the element is found.
4. Calculate the total number of comparisons.

Consider the sorted list

10 18 24 31 36 40 47 55 61 68 74 82 90 95 100

The block size is **5**.

Tasks

Construct the index table.

Search for **68**.

Search for **25**.

Count the comparisons in both cases.

Problem 3

A company stores employee IDs in sorted order.

101 108 115 122 130 138 145 151 160 169 177 185 192 201 214 225

The block size is **4**.

Questions

- Construct the index table.
- Search for Employee ID **177**.
- Search for Employee ID **200**.
- Explain why only one block is searched after consulting the index.